

ASSESSMENT TOOL 1- DASHBOARD CREATION

Course Code: DSA0504	Course Title: Query Processing for Data Science
CO Assessed: CO5	Assessment: Tool 1– Dashboard Creation
Weightage: 40	Total Marks: 40 Marks
CO5 Statement: Create meaningful visualizations using Pandas and Matplotlib, including charts,distributions, relationships, and interactive visual representations for effective communication.	
Student Name:J.Subhaashree	Reg. No.:192424008
Section / Batch:2024	Date of Submission:26/8/26
Faculty In-charge: Dr. R.Mahesh Muthulakshmi	Project Mode: Individual Submission

1)Optimization of Placement Dashboard Query for Faster Execution

Aim

To identify and apply database optimization techniques to reduce the execution time of a placement dashboard query involving Student, Department, Company, and Placement tables.

Problem Statement

A placement dashboard executes a query that joins the Student, Department, Company, and Placement tables. Since the query takes several seconds to execute, optimization techniques are required to improve its performance and reduce response time.

Optimization Strategies

1. Create Appropriate Indexes

Indexes should be created on columns that are frequently used in JOIN, WHERE, and ORDER BY clauses. For example, indexes can be created on student ID, department ID, and company ID.

```
CREATE INDEX idx_student_dept ON Student(Dept_ID);  
CREATE INDEX idx_placement_company ON Placement(Company_ID);
```

Indexes allow the database to find required records faster without scanning the entire table.

2. Optimize the SQL Query

Only the required columns should be selected instead of using SELECT *. Filtering should also be performed as early as possible to reduce the number of rows processed during joins.

Example:

```
SELECT s.Student_Name, d.Department_Name,  
       c.Company_Name, p.Package  
FROM Student s  
JOIN Department d ON s.Dept_ID = d.Dept_ID  
JOIN Placement p ON s.Student_ID = p.Student_ID  
JOIN Company c ON p.Company_ID = c.Company_ID  
WHERE p.Status = 'Placed';
```

3. Use Query Execution Plans

The EXPLAIN or EXPLAIN ANALYZE command can be used to examine how the database executes the query.

```
EXPLAIN ANALYZE  
SELECT ...
```

It helps identify problems such as full table scans, inefficient joins, and missing indexes. The query can then be modified based on the execution plan.

4. Use Materialized Views or Summary Tables

If the dashboard repeatedly performs the same complex joins, a materialized view can store the precomputed results. The dashboard can retrieve the prepared data instead of executing all joins each time.

Expected Result

By applying these optimization techniques, the number of rows processed and the amount of database work can be reduced. This results in faster query execution and improved dashboard response time.

DASHBOARD



Conclusion

The execution time of the placement dashboard can be reduced by using proper indexing, SQL query optimization, execution-plan analysis, and materialized views or summary tables. These techniques improve database performance and provide faster access to placement information.

2) Join Selection for Large-Scale E-Commerce Dashboard

For an e-commerce dashboard joining Customer, Orders, and Products tables, the choice of join depends on the required output. For large datasets, INNER JOIN is generally preferred when only customers with valid orders and products need to be displayed.

Comparison of Joins

Join Type	Description	Suitability
INNER JOIN	Returns only matching records from both tables.	Best for large datasets when only valid purchase records are required.
LEFT JOIN	Returns all records from the left table and matching records from the right table.	Useful when customers with no orders must also be shown.
RIGHT JOIN	Returns all records from the right table and matching records from the left table.	Less commonly used; can usually be replaced by rearranging tables.
FULL OUTER JOIN	Returns matching and non-matching records from both tables.	More data is processed, so it may be less efficient for large datasets.

Preferred Join: INNER JOIN

For this application, I would prefer INNER JOIN for large datasets because:

1. It returns only matching customer, order, and product records.
2. It reduces the number of rows that need to be processed.
3. It avoids unnecessary records such as customers who have never purchased anything.
4. With proper indexes on Customer_ID, Order_ID, and Product_ID, query performance can be improved further.

Example

```
SELECT c.Customer_Name,
       p.Product_Name,
       o.Quantity,
       o.Order_Date
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Products p
    ON o.Product_ID = p.Product_ID;
```

DASHBOARD



Conclusion

INNER JOIN is preferred for large datasets when the dashboard needs only customers who have made purchases. It processes fewer records and, when combined with suitable indexes, provides better query performance. However, if the dashboard must also show customers with no purchases, a LEFT JOIN would be more appropriate.

Query Optimization for University Academic and Placement Dashboard

Aim

To optimize a department-wise academic and placement query for a database containing 10 million student records and reduce its execution time.

Problem

A university dashboard takes 18 seconds to generate a department-wise academic and placement report from 10 million records. The query uses multiple joins, selections, aggregations, and sorting.

Optimization Strategies

1. Query Decomposition – Divide the complex query into smaller operations and process filtered data before joining.
2. Selection & Projection Pushdown – Apply WHERE conditions early and retrieve only required columns to reduce the number of rows and data processed.
3. Join Ordering & Indexing – Join smaller filtered relations first and create indexes on frequently used join and filtering columns.
4. Cost-Based Optimization & Execution Plans – Use EXPLAIN ANALYZE to identify expensive joins, scans, sorting, and missing indexes, and allow the optimizer to select a low-cost execution plan.
5. Materialized Views & Parallel Processing – Use precomputed materialized views for frequently requested reports and parallelize large scans, joins, aggregations, and sorting across CPU cores.

Original Query

```
SELECT d.Dept_Name,  
       COUNT(DISTINCT s.Student_ID) AS Total_Students,  
       COUNT(p.Student_ID) AS Placed_Students,  
       AVG(p.Package) AS Average_Package  
FROM Student s  
JOIN Department d  
  ON s.Dept_ID = d.Dept_ID  
JOIN Placement p  
  ON s.Student_ID = p.Student_ID  
WHERE s.Academic_Year = '2025-26'  
      AND p.Status = 'Placed'  
GROUP BY d.Dept_Name  
ORDER BY Placed_Students DESC;
```

Optimized Query

```
WITH FilteredStudents AS (  
  SELECT Student_ID, Dept_ID  
  FROM Student  
  WHERE Academic_Year = '2025-26'  
)  
FilteredPlacements AS (  
  SELECT Student_ID, Package  
  FROM Placement  
  WHERE Status = 'Placed'  
)  
SELECT d.Dept_Name,  
       COUNT(DISTINCT s.Student_ID) AS Total_Students,  
       COUNT(p.Student_ID) AS Placed_Students,  
       AVG(p.Package) AS Average_Package  
FROM FilteredStudents s  
JOIN Department d  
  ON s.Dept_ID = d.Dept_ID
```



```
LEFT JOIN FilteredPlacements p
  ON s.Student_ID = p.Student_ID
GROUP BY d.Dept_Name
ORDER BY Placed_Students DESC;
```

Index Queries

```
CREATE INDEX idx_student_year_dept
ON Student(Academic_Year, Dept_ID, Student_ID);
CREATE INDEX idx_placement_student_status
ON Placement(Student_ID, Status);
CREATE INDEX idx_department_id
ON Department(Dept_ID);
```

Execution Plan Query

```
EXPLAIN ANALYZE
SELECT ...;
```

This identifies full table scans, expensive joins, sorting operations, and missing indexes.

Materialized View Query

```
CREATE MATERIALIZED VIEW Department_Placement_Summary AS
SELECT Dept_ID,
       COUNT(DISTINCT Student_ID) AS Total_Students,
       COUNT(*) AS Placed_Students,
       AVG(Package) AS Average_Package
FROM Placement
WHERE Status = 'Placed'
GROUP BY Dept_ID;
```

Then the dashboard can use:

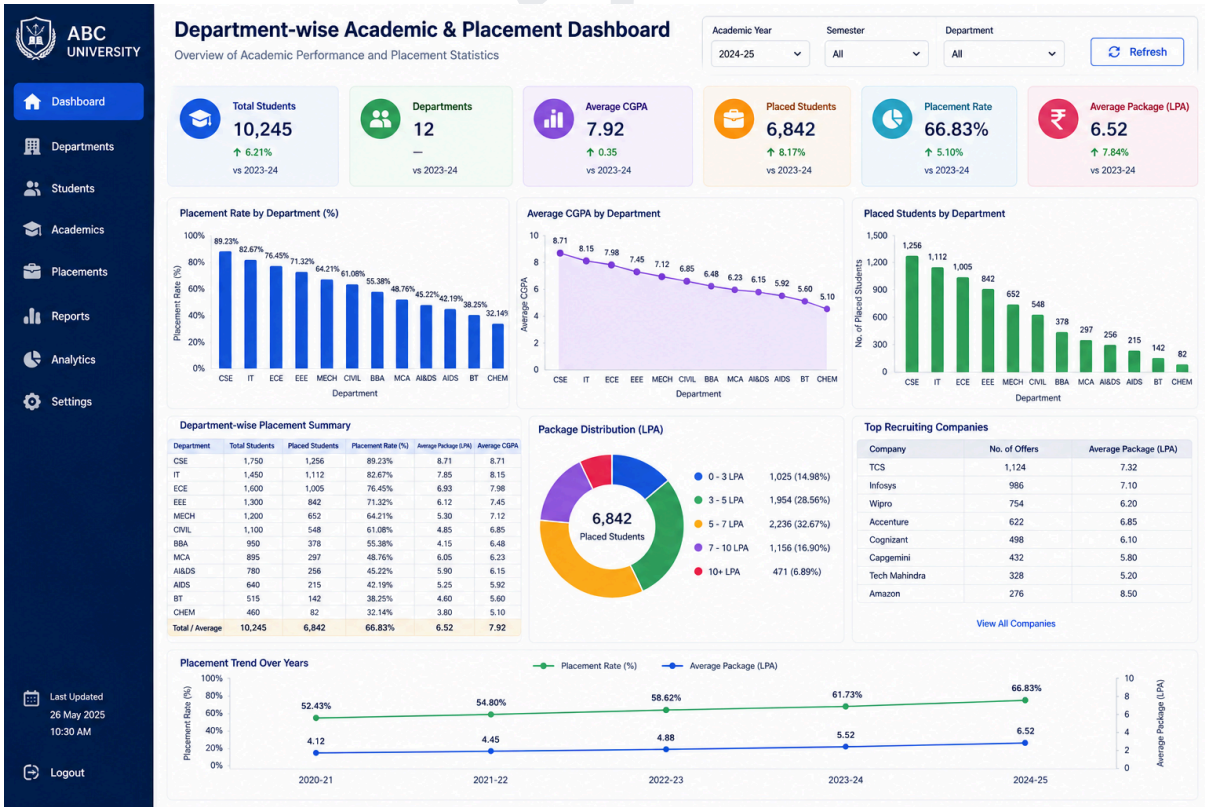
```
SELECT *
FROM Department_Placement_Summary
ORDER BY Placed_Students DESC;
```

Experimental Comparison

Run both queries several times under the same conditions and compare execution time.

Metric	Original	Optimized
Execution Time	18 sec	Expected lower
Rows Processed	Very High	Reduced
CPU Usage	High	Lower
Disk I/O	High	Lower
Full Table Scan	Possible	Reduced

DASHBOARD



The optimized query should provide faster dashboard response by reducing unnecessary data processing and improving join, filtering, indexing, and aggregation efficiency.

192424008